

```
temp_files_to_clean.remove("temp_1.log")

# Attempt to remove a file that might not be there (discard - safe)
temp_files_to_clean.discard("temp_99.tmp") # No error raised

print(f"Remaining files to clean: {temp_files_to_clean}") # {'temp_2.bak'}
```

frozenset (Immutable version of set)

A frozenset is an immutable set. Once created, you cannot add or remove elements. This makes it hashable, so it can be used as a **key in a dictionary** or as an **element in another set**.

Python

```
# Real-time Example: Using a fixed set of permissions as a dictionary key
# This can't be done with a regular mutable set.
permissions = frozenset(["read", "write"])

access_levels = {
    permissions: "Administrator",
    frozenset(["read"]): "Guest"
}

print(f"Access level for {list(permissions)}: {access_levels[permissions]}")
```

6. Modules and Packages

6.1 Modules

What is a Module?

A module is essentially a **single Python file** (with the .py extension) that contains definitions, statements, and functions. Modules allow you to logically organize your Python code, making it reusable and easier to maintain.

Real-time Analogy: Think of a module as a specialized **tool kit** (e.g., a math tool kit or a file-handling tool kit). When you need to perform a specific task, you import the relevant tool kit instead of rewriting the tools yourself.

Creating a Module

Creating a module is as simple as saving code in a .py file.

Example: data_processor.py Let's create a module for processing sales data.

Python

```
# --- File: data_processor.py ---

# Global variable for configuration
TAX_RATE = 0.05
```

```
def calculate_total(price, quantity):
    """Calculates the subtotal (price * quantity)."""
    return price * quantity

def apply_tax(subtotal):
    """Applies the global TAX_RATE to the subtotal."""
    return subtotal * (1 + TAX_RATE)

def format_currency(amount):
    """Formats a number as a currency string."""
    return f"${amount:,.2f}"

# Initial print statement runs only once when imported
print("Data Processor Module Initialized.")
```

Importing Modules: import and from...import

There are two primary ways to bring the functionalities of a module into your current script:

1. **import module_name:** Imports the entire module. You must use the module name as a prefix to access its contents (module_name.function()).
2. **from module_name import item1, item2:** Imports only specific items (functions, variables, classes) directly into your current namespace. You can use them without the module prefix.

Example: main_script.py

Python

--- File: main_script.py ---

1. Importing the entire module

```
import data_processor
# Accessing using the prefix
subtotal = data_processor.calculate_total(price=100.00, quantity=3)
taxed_total = data_processor.apply_tax(subtotal)
final_output = data_processor.format_currency(taxed_total)
```

```
print(f"\n--- Using 'import data_processor' ---")
print(f"Tax Rate: {data_processor.TAX_RATE}")
print(f"Final Total: {final_output}")
```

2. Importing specific items (avoids prefix)

```
from data_processor import calculate_total, TAX_RATE as SALES_TAX # Renaming TAX_RATE
```

```
print(f"\n--- Using 'from...import' ---")
# The variable is now named SALES_TAX in this script
print(f"Sales Tax: {SALES_TAX}")
# The function can be called directly
quick_subtotal = calculate_total(50, 2)
print(f"Quick Subtotal: {quick_subtotal}")
```

Module Search Path (sys.path)

When you use an import statement, the Python interpreter searches for the requested module in a specific order:

1. **Current Directory:** The directory where the executing script resides.
2. **PYTHONPATH Directories:** Directories specified in the PYTHONPATH environment variable.
3. **Standard Library Directories:** Directories containing built-in Python modules (like os, sys, math).
4. **Site-packages Directory:** The location for third-party modules (like requests, numpy) installed via pip.

This path is stored in the sys.path list.

Real-time Example: Debugging import errors.

Python

```
# --- Example: Viewing the Search Path ---
```

```
import sys
```

```
print("\n--- Module Search Path (sys.path) ---")
```

```
for path in sys.path:
```

```
    print(path)
```

```
# If your module isn't found, you typically need to:
```

```
# 1. Ensure it's in the same directory.
```

```
# 2. If it's elsewhere, you can temporarily add its path:
```

```
# sys.path.append("/path/to/my/custom/modules")
```

```
# import my_module # This will now work
```

The dir() function

The built-in dir() function returns a list of names (variables, functions, classes) defined in a module or currently available in the scope. It is useful for **discovery** and **introspection**.

Python

```
# Using dir() on our imported module
```

```
import data_processor
```

```
print("\n--- Contents of data_processor module (dir()) ---")
```

```
# Prints: ['TAX_RATE', '__builtins__', ..., 'apply_tax', 'calculate_total', 'format_currency']
```

```
print(dir(data_processor))
```

```
# Using dir() without arguments shows the current scope
```

```
# print(dir()) # Shows items imported/defined in main_script.py
```

Using __name__ == '__main__'

Every Python module has a built-in special variable called __name__.

- When the file is executed directly by the interpreter, Python sets __name__ to the string '__main__'.
- When the file is imported as a module into another script, Python sets __name__ to the module's name (e.g., 'data_processor').

This condition is used to define code that should **only run when the script is executed directly**, not when it is imported. This is crucial for adding unit tests or demonstration code to a module without running them every time it's imported.

Example: Enhancing data_processor.py

Python

```
# --- File: data_processor.py (Revisited) ---
# ... (functions remain the same) ...

# Test/Demonstration Code (runs only when this file is executed directly)
if __name__ == '__main__':
    print("\n--- Running module as a standalone script (Demo) ---")

    # Simple test case
    item_price = 45.99
    item_qty = 2

    subtotal = calculate_total(item_price, item_qty)
    final = apply_tax(subtotal)

    print(f'Item Price: {format_currency(item_price)}')
    print(f'Subtotal: {format_currency(subtotal)}')
    print(f'Total with Tax ({TAX_RATE*100}%): {format_currency(final)}')
```

6.2 Packages

What is a Package?

A package is a way to organize related modules into a directory hierarchy.

- A **module** is a single file (.py).
- A **package** is a directory containing related modules and a special file called `__init__.py`.

Real-time Analogy: A package is like a **library** (e.g., a "Financial Analysis" library) which contains specialized tool kits (modules) like `calculations.py`, `reporting.py`, and `data_fetching.py`.

Creating a Package (Role of `__init__.py`)

A directory becomes a Python package when it contains a file named `__init__.py`. This file can be empty, but its presence signals to Python that the directory should be treated as a package.

In modern Python (3.3+), a directory can be considered a namespace package even without `__init__.py`, but including it is still standard practice for simple packages.

Example: Creating a reporting Package

```
/my_project
├── main.py
├── reporting/          <-- Package Directory
│   ├── __init__.py    <-- Identifies 'reporting' as a package
│   ├── report_generator.py <-- Module 1
│   └── email_sender.py  <-- Module 2
```

`report_generator.py`

Python

```
def generate_summary_report(data):
    return f'Summary: Processed {len(data)} records.'
```

email_sender.py

Python

```
def send_email(recipient, subject, body):  
    return f"Email sent to {recipient} with subject: '{subject[:20]}...'"
```

Importing from Packages

To use modules and functions within a package, you use the dot (.) notation to specify the package and module hierarchy.

Example: main.py

Python

--- File: main.py ---

```
# 1. Importing a specific function from a module inside the package  
from reporting.report_generator import generate_summary_report
```

```
# 2. Importing the entire module from the package  
import reporting.email_sender
```

```
data = [1, 2, 3, 4, 5, 6]
```

```
# Using the function imported directly  
summary = generate_summary_report(data)  
print(f"Report Output: {summary}")
```

```
# Using the function imported via the module path  
email_result = reporting.email_sender.send_email(  
    recipient="admin@corp.com",  
    subject="Weekly Sales Report",  
    body="Attached is the detailed report..."  
)  
print(f"Email Status: {email_result}")
```

7. Object-Oriented Programming (OOP) - The Core Pillars

7.1 Classes and Objects

Defining a Class (The Blueprint)

A **Class** is a blueprint or a template for creating objects. It defines a set of attributes (data/variables) and methods (functions) that the created objects will possess.

Real-time Example: A Vehicle class.

Creating an Object (The Instance)

An **Object** (or instance) is a concrete entity built from the class blueprint. Each object has its own unique set of attribute values.